

Week 5, Lecture 1: Synthetic Data Simulation

Instructor notes – Part 1: prep reading, Part 2: podium cheat sheet

This single file has two parts, both instructor-only (not a student handout). **Part 1** is read beforehand – full worked examples and the reasoning behind the flow. **Part 2** (after the page break, large print) is the podium copy.

Source material: Bolker Ch. 5 intro and 5.2 (stochastic simulation, p. 197-201, the single-group linear/hyperbolic examples). Also mined: legacy `LECTURE3.Rmd` (“Data Generating Models”) – its “why simulate fake data” list, deterministic/stochastic decomposition, and the tree volume goodness-of-fit example are well-tested and reused below with modernized R code; its power-analysis monitoring-program example is flagged for tomorrow’s Lab 4 rather than rebuilt here, since fully coding it up is more than today’s lecture has time for.

Where this sits in the week

- Last week (Week 4): combinatorics, random variables, probability distributions (pmf vs. pdf), expectation and variance, and the bestiary of distributions – students now have a toolbox of named distributions and know how to compute their moments. (This lecture was originally slotted for Week 4, Wednesday, but the distributions material ran through that session, so it moved here.)
- Last Tuesday: no lab Acts – lab time was final project proposal pitches instead.
- **Today (Monday):** those distributions stop being static objects to describe and become the *stochastic* half of something you can build and run – a full **data-generating model**. This is direct setup for tomorrow’s Lab 4 (“Data generating models”), which builds a full power-analysis simulation; today stays conceptual and example-driven, leaving the deep hands-on coding for the lab.
- Wednesday: intro to likelihood – the same distributions, run in reverse (data fixed, parameters varying).

Rationale for the flow

Bolker’s own framing for Chapter 5 is useful to state explicitly: simulation is *forward* modeling (pick a model and parameters, predict data), while the rest of the course is mostly about *inverse* modeling (start from data, work backward to parameters). Students haven’t done any inverse modeling yet (that starts Wednesday, with likelihood), so today’s forward-modeling framing is genuinely their first exposure to “build a model and run it,” not a contrast with

something they already know how to do – keep the framing simple rather than assuming the inverse-modeling contrast lands yet.

Keep today to one clean running example (the deterministic + stochastic decomposition, worked twice – once as pure demonstration, once as a goodness-of-fit check against real-looking data) rather than touring many models. The payoff is conceptual: students should leave able to name *why* you’d build a synthetic data generator, not just how to call `rnorm()`.

Part A: Why simulate data? (~8 min)

Open with the Popperian framing from earlier in the semester:

THEORY/QUESTION → CANDIDATE MODEL(S) → confront with DATA → COMPARE → repeat

A synthetic data generator *is* a candidate model, made concrete enough to run. If you can write the computer program that generates data under your hypothesis, you’ve forced every assumption into the open – there’s nowhere for a vague verbal hypothesis to hide.

Four reasons this matters (ask students to guess some before listing):

- **Validate model fitting.** Before trusting an estimation procedure on real data (where you never know the true answer), run it on data you generated yourself – you *know* the true parameters, so you can check whether the method recovers them.
- **Power analysis.** Simulate data under a hypothesized effect and a proposed sampling design, then ask how often the analysis would actually detect the effect. Tells you whether a study is worth running before you run it.
- **Improved understanding.** Chaining a deterministic function to a stochastic layer and watching what comes out builds intuition for a model’s behavior that staring at the equation alone doesn’t – you see the consequences of your assumptions, including ones you didn’t realize you’d made.
- **Forecasting.** A validated data-generating model can be run forward under new conditions (new covariate values, future time points) to predict what hasn’t been observed yet.

Sticking point to flag: “fake data” sounds like something you’d never show a reviewer, but the point isn’t to replace real data – it’s a diagnostic tool you use *before and alongside* real data, the same way you’d sanity-check any other model output.

Part B: Anatomy of a data-generating model (~10-12 min)

Bolker’s running distinction: a **deterministic component** (the “signal,” giving an expected value) plus a **stochastic component** (the “noise,” a probability distribution wrapped around that expected value). Last week’s distributions lectures supplied exactly the vocabulary needed for the second half of this.

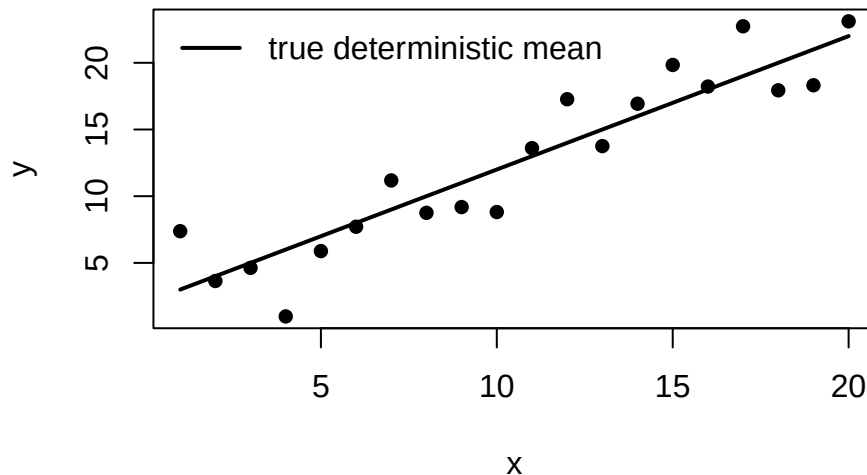
Simplest case – linear regression’s data-generating model:

$$\hat{\mu}_i = a + bx_i \quad Y_i \sim \text{Normal}(\hat{\mu}_i, \sigma)$$

```
x <- 1:20                                     # predictor: e.g. nitrogen concentration
a <- 2; b <- 1
mu <- a + b * x                               # deterministic component

set.seed(1001)
y <- rnorm(length(x), mean = mu, sd = 2)      # stochastic component layered on top

plot(x, y, pch = 16, xlab = "x", ylab = "y")
lines(x, mu, lwd = 2)
legend("topleft", legend = "true deterministic mean", lwd = 2, bty = "n")
```



Point out explicitly: the deterministic function gives the *expected value* of Y_i at each x_i , and for the normal that expected value is its mean parameter – this is the same $Y_i \sim \text{Normal}(\mu, \sigma^2)$ notation from last week, just with μ replaced by a function of x instead of a constant. For now the deterministic function always models the expected value; the modeling-variance week (Nov. 2) generalizes to functions for other parameters.

A case where the components don’t just add together. Not every model can be written as “deterministic plus noise” – sometimes the noise is not additive or symmetric, and its spread depends on the expected value the function gives. Example: a hyperbolic decline in fecundity with population density, with Poisson-distributed counts (rather than negative binomial, so this stays inside last week’s distributions and doesn’t require introducing overdispersion):

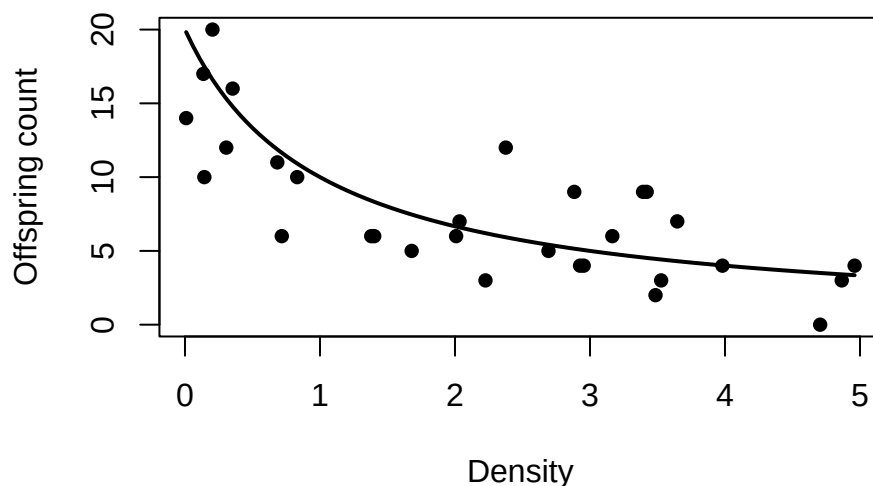
```

dens <- runif(30, min = 0, max = 5)           # population density, sampled from the fie
a <- 20; b <- 1
mu_det <- a * b / (b + dens)                  # deterministic: hyperbolic decline

set.seed(1001)
fecundity <- rpois(30, lambda = mu_det)        # stochastic: Poisson counts around that m

plot(dens, fecundity, pch = 16, xlab = "Density", ylab = "Offspring count")
curve(a * b / (b + x), add = TRUE, lwd = 2)

```



Sticking point to flag: the Poisson's rate parameter λ is *simultaneously* its mean and variance (last week's equidispersion point) – so as the deterministic curve declines, the *spread* of the noise shrinks right along with it. That's visible in the plot (points hug the curve more tightly at low density) and is a real, checkable consequence of choosing this distribution, not an artifact.

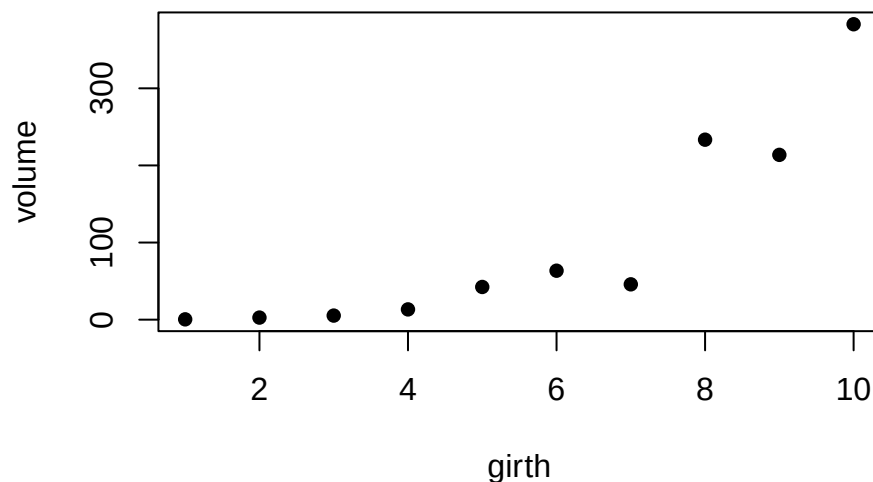
Part C: Using synthetic data as a goodness-of-fit check (~10-12 min)

Worked example (adapted from legacy LECTURE3.Rmd, tree girth/volume): imagine a small set of “real” measurements.

```

tree_data <- data.frame(girth = seq(1, 10, length.out = 10),
                        volume = c(0.4, 2.7, 5.3, 13.3, 42.4,
                                   63.5, 45.8, 233.4, 213.7, 383.1))
plot(volume ~ girth, data = tree_data, pch = 16)

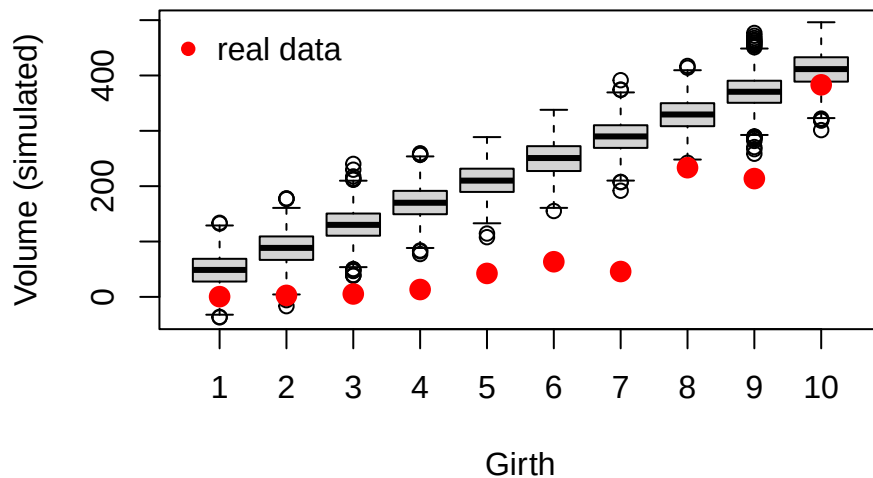
```



The question a data-generating model can answer: could a *specific, fully-specified* hypothesis have plausibly produced this data? Propose one: $\text{Volume}_i \sim \text{Normal}(a + b \cdot \text{Girth}_i, \sigma)$, with $a = 10$, $b = 40$, $\sigma = 31$. Simulate many datasets under that exact hypothesis and see whether the real data falls inside the plausible range:

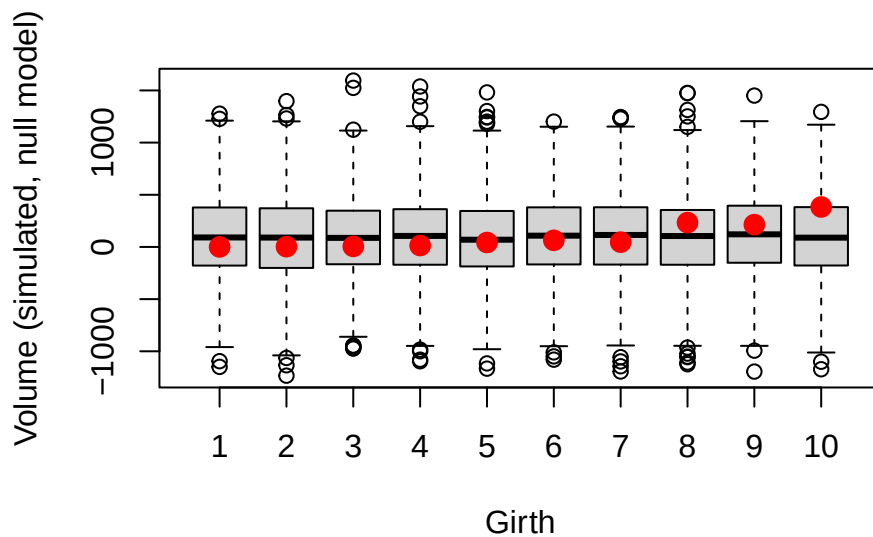
```
n_sim <- 1000
n_obs <- nrow(tree_data)
sim_volume <- replicate(n_sim, rnorm(n_obs, mean = 10 + 40 * tree_data$girth, sd = 31))

boxplot(t(sim_volume), names = tree_data$girth,
        xlab = "Girth", ylab = "Volume (simulated)")
points(seq_len(n_obs), tree_data$volume, pch = 16, col = "red", cex = 1.5)
legend("topleft", legend = "real data", pch = 16, col = "red", bty = "n")
```



The real data (red) sits well inside the simulated spread – this particular hypothesis is at least *plausible*. Contrast with an obviously wrong hypothesis (flat mean, no girth effect):

```
sim_null <- replicate(n_sim, rnorm(n_obs, mean = 100, sd = 400))
boxplot(t(sim_null), names = tree_data$girth,
        xlab = "Girth", ylab = "Volume (simulated, null model)")
points(seq_len(n_obs), tree_data$volume, pch = 16, col = "red", cex = 1.5)
```



Both technically “contain” the red points somewhere in their range (the null’s variance is

huge), but the pattern is obviously worse – worth asking students *why* a wide-enough null can always “fit” in this loose visual sense, and what that implies about needing sharper comparison tools later in the course (a preview of Wednesday’s intro to likelihood and Weeks 6-7’s likelihood-based model comparison, which replaces “eyeball it” with a number).

Sticking point to flag: this is a *plausibility* check, not proof – “the data are consistent with this model” is a much weaker (and more honest) claim than “this model is correct.” That distinction is the same corroborate-vs-prove vocabulary from the semester’s opening Popperian framing.

Take-home message

- A data-generating model = a deterministic function (the signal) plus a probability distribution (the noise) – last week’s distributions supply the second half.
- Simulating data is useful before you ever touch real data: it validates fitting procedures, powers a power analysis, builds intuition, and lets you forecast.
- Comparing real data against simulated replicates is a visual goodness-of-fit / plausibility check – a first, informal version of the model comparison tools coming later in the semester.

Roadmap tie-in

- Tomorrow (Lab 4, “Data generating models”): build a full power-analysis simulation from scratch (a monitoring-program example – how large a sample size is needed to reliably detect a population decline), going well beyond today’s conceptual introduction.
- Wednesday (intro to likelihood) and Weeks 6-7 (maximum likelihood, model comparison): replace today’s “eyeball the boxplot” goodness-of-fit check with a quantitative one.

Discussion seed (if time remains)

Ask: *if you were designing a monitoring study for a species you study, what would your data-generating model need to include – what’s the deterministic part (the ecological signal you actually care about) and what’s the stochastic part (sampling error, detection error, natural variability)? Would more than one source of randomness need to be chained together?*

PART 2 – PODIUM CHEAT SHEET

Podium copy – terse on purpose. Full derivations, rationale, and code: see Part 1 above.

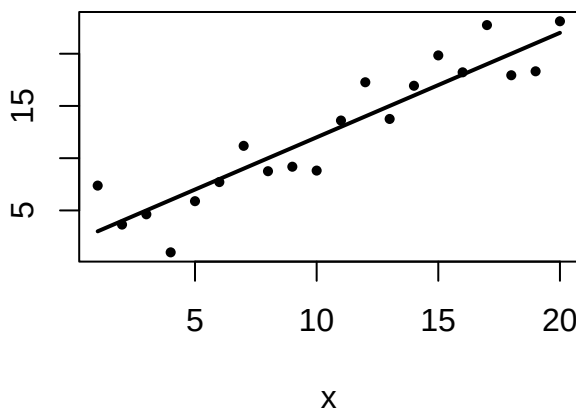
0. Open (~2 min)

- Recap Popperian loop: THEORY $\rightarrow$ MODEL $\rightarrow$ DATA $\rightarrow$ COMPARE $\rightarrow$ repeat – a synthetic data generator makes a candidate model concrete enough to *run*

1. Why simulate? (~8 min)

- **Validate** model-fitting (you know the true answer, so you can check recovery)
- **Power analysis** (how often would this design detect a real effect?)
- **Understanding** (chaining functions reveals consequences of assumptions)
- **Forecasting** (run the validated model forward under new conditions)
- Not a replacement for real data – a diagnostic used alongside it

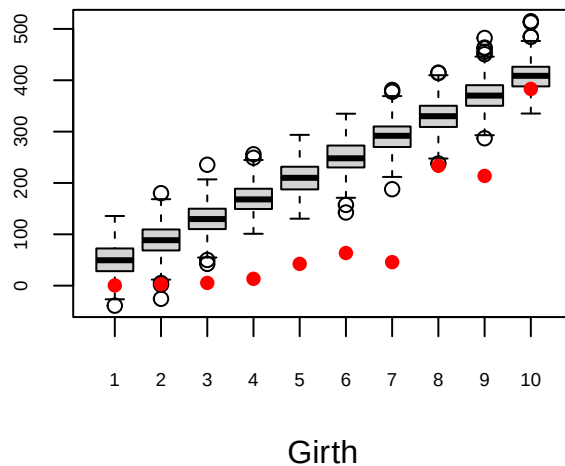
2. Anatomy: deterministic + stochastic (~10-12 min)



- $\hat{\mu}_i = a + bx_i$ (deterministic/“signal”) + $Y_i \sim \text{Normal}(\hat{\mu}_i, \sigma)$ (stochastic/“noise”)
- Deterministic function = expected value of Y (last week’s μ , now a function of x)

- **Sticking point:** not always additive – a hyperbolic $\times$ Poisson example shrinks noise right along with the shrinking mean (equidispersion)

3. Synthetic data as a goodness-of-fit check (~10-12 min)



- Propose a fully-specified hypothesis, simulate many datasets under it, overlay real data – does it fall inside the plausible range?
- **Sticking point:** plausibility $\neq$ proof – “consistent with” is weaker than “correct” (corroborate, don’t prove)

4. Roadmap (~2 min)

- Tomorrow (Lab 4): full power-analysis build, from scratch
- Wednesday + Weeks 6-7: quantitative model comparison replaces “eyeball the boxplot”

Discussion (if time)

- *Designing a monitoring study for your own system: what’s the deterministic part? What’s the stochastic part? More than one source of randomness to chain together?*